A2 Analysis and Report

Ishan Lakhotia

4.1 Correctness Analysis

In A2, the decryption program for both serial and parallel is considered correct if it produces plaintext decryption of the given ciphertext, where every word(s) exists in the provided dictionary. Also, if it completes execution without crashes or undefined behaviour. In addition, the output of the generated words from the decryption should be the same and consistent between both the serial and parallel program.

To verify the correctness, I designed a testing procedure using test cases where the expected plaintext was known in advance. I began with smaller ciphertexts containing usually one encrypted word so that the expected permutations could be determined manually. After confirming correctness on these minimal cases, I increased difficulty by testing longer and more difficult ciphertexts, often ciphertexts containing multiple encrypted words and spaces. I also then tested encrypted words that were not in the American-English dictionary, in which case the correct behaviour is to produce no valid plaintext output. In each case, the ciphertext.txt file was used as I first did an ./a2encrypt where it writes the generated ciphertext to ciphertext.txt. Then called both the serial implementation (./a2decrypt_serial) and the parallel implementation (./a2decrypt) to ensure identical output behaviour, and make sure the output is correct..

I performed memory correctness checks using Valgrind on the serial program, since it is easier to check if my code actually has leaks in the single-process version. After ensuring that all dictionary memory was freed at program termination. The parallel program had memory leaks but that was due to MPI, not my specific code as all memory allocated was freed correctly.

This combination of manually verifiable test cases and more difficult ones provides confidence that both the serial and parallel implementations are correct.

Test Cases:

Simple Tests:

To run the simple tests, make sure you are in the A2 directory. Then do ./simpleTests.sh in the command line/terminal **(*Note you might have to do chmod +x)**. This will create and write the output to outputs/simpleTest.txt

Tests:

***Please note that the encryption dictionary will change or get shuffled after every run. This means that in the parallel output, the rank will probably be different due to the encryption/ciphertext changing each run. This means that different processes or ranks will find the proper decryption of the cipher text after each run. Also the order of the way decrypted words are shown for parallel might differ in each run through as well. However, the decrypted words will always be the same/consistent**

Test #	Expected Output
1	Simple Test 1 Input: first Input dictionary: first Encryption dictionary: rfits Encrypted text: rfits Written to ciphertext.txt Serial Output: rank 0: rifts rank 0: first Parallel Output: rank 0: rifts rank 1: first
2	Simple Test 2 Input: task Input dictionary: task Encryption dictionary: kast Encrypted text: kast Written to ciphertext.txt Serial Output: rank 0: task Parallel Output: rank 3: task
3	Simple Test 3 Input: begin Input dictionary: begin Encryption dictionary: negib Encrypted text: negib Written to ciphertext.txt Serial Output: rank 0: begin rank 0: being rank 0: binge Parallel Output:

	rank 4: begin rank 4: being rank 4: binge
4	Simple Test 4 Input: iceboxes Input dictionary: iceboxs Encryption dictionary: xceobis Encrypted text: xceobies Written to ciphertext.txt Serial Output: rank 0: iceboxes Parallel Output: rank 5: iceboxes
5	Simple Test 5 Input: the cat Input dictionary: theca Encryption dictionary: chtea Encrypted text: cht eac Written to ciphertext.txt Serial Output: rank 0: the cat rank 0: the act rank 0: eta che rank 0: eat che Parallel Output: rank 2: the cat rank 2: the act rank 3: eta che rank 3: eat che

Longer Tests:

To run the longer tests, make sure you are in the A2 directory. Then do ./longerTests.sh in the command line/terminal **(*Note you might have to do chmod +x)**. This will create and write the output to outputs/longerTests.txt

Tests:

Test #	Expected Outcome
1	Long Test 1 Input: cat in the hat

	Input dictionary: catinhe Encryption dictionary: neahcti Encrypted text: nea hc ati tea Written to ciphertext.txt Serial Output: rank 0: nat ci the hat rank 0: nit ca the hit rank 0: nit ac the hit rank 0: ant he tic int rank 0: ant ch tie int rank 0: ant eh tic int rank 0: han ec nit ian rank 0: hat ci ten eat rank 0: hie an etc tie rank 0: hie nc eta tie rank 0: hie na etc tie rank 0: can eh nit ian rank 0: can he nit ian rank 0: can ie nth tan rank 0: cat ni the hat rank 0: cat hi ten eat rank 0: cat in the hat rank 0: che ni eta the rank 0: che in eta the rank 0: cia nh ate tia rank 0: tia nh ace cia rank 0: ian ec nth tan rank 0: ice nh eat ace Parallel Output: rank 2: ant he tic int rank 2: ant ch tie int rank 2: ant eh tic int rank 3: han ec nit ian rank 3: hat ci ten eat rank 4: can eh nit ian rank 4: can he nit ian rank 4: can ie nth tan rank 4: cat ni the hat rank 4: cat hi ten eat rank 0: nat ci the hat rank 5: tia nh ace cia rank 6: ian ec nth tan rank 6: ice nh eat ace rank 0: nit ca the hit rank 0: nit ac the hit rank 4: cat in the hat rank 4: che ni eta the rank 3: hie an etc tie rank 3: hie nc eta tie rank 3: hie na etc tie rank 4: che in eta the rank 4: cia nh ate tia

2	Input: fort is big Input dictionary: fortisbg Encryption dictionary: sgofirtb Encrypted text: sgof ir tib Written to ciphertext.txt Serial Output: rank 0: soft ir big rank 0: sort if big rank 0: sift or gob rank 0: sift or bog rank 0: stir of gob rank 0: stir of bog rank 0: stir ob fog rank 0: gobs ir fit rank 0: gobs it fir rank 0: gift rs orb rank 0: gift rb sro rank 0: gift or sob rank 0: gift os rob rank 0: gist or fob rank 0: gist of rob rank 0: gist ob for rank 0: gist rb fro rank 0: girt os fob rank 0: girt of sob rank 0: gris of bot rank 0: gris ot fob rank 0: grit os fob rank 0: grit of sob rank 0: orbs it fig rank 0: orbs it gif rank 0: fogs ir bit rank 0: fogs it rib rank 0: fort is big rank 0: fobs it rig rank 0: figs or bot rank 0: figs ot rob rank 0: figs ob tor rank 0: figs ob rot rank 0: fist or gob rank 0: fist or bog rank 0: firs ot gob rank 0: firs ot bog rank 0: firs ob tog rank 0: firs ob got rank 0: fits or gob rank 0: fits or bog rank 0: fibs or tog rank 0: fibs or got rank 0: frog is bit rank 0: frog tb its rank 0: frog bi tbs rank 0: rots if big rank 0: robs it fig rank 0: robs it gif

	rank 0: robt is fig rank 0: robt is gif rank 0: rigs of bot rank 0: rigs ot fob rank 0: rift os gob rank 0: rift os bog rank 0: ribs of tog rank 0: ribs of got rank 0: ribs ot fog rank 0: togs if rib rank 0: togs ir fib rank 0: togs rb fri rank 0: togs br fbi rank 0: tors if big rank 0: trig of sob rank 0: trig os fob rank 0: bogs ir fit rank 0: bogs it fir rank 0: borg ft ifs rank 0: borg if sit rank 0: borg is fit rank 0: bits or fog rank 0: bros it gif rank 0: bros it fig rank 0: brig of sot rank 0: brit os fog Parallel Output: rank 0: soft ir big rank 0: sort if big rank 6: togs if rib rank 6: togs ir fib rank 6: togs rb fri rank 6: togs br fbi rank 7: bogs ir fit rank 7: bogs it fir rank 6: tors if big rank 7: borg ft ifs rank 7: borg if sit rank 7: borg is fit rank 1: gobs ir fit rank 1: gobs it fir rank 7: bits or fog rank 7: bros it gif rank 7: bros it fig rank 7: brig of sot rank 7: brit os fog rank 2: orbs it fig rank 2: orbs it gif rank 3: fogs ir bit rank 3: fogs it rib rank 3: fort is big rank 3: fobs it rig rank 3: figs or bot rank 3: figs ot rob rank 3: figs ob tor

	rank 3: figs ob rot rank 3: fist or gob rank 3: fist or bog rank 3: firs ot gob rank 3: firs ot bog rank 3: firs ob tog rank 3: firs ob got rank 3: fits or gob rank 3: fits or bog rank 3: fibs or tog rank 3: fibs or got rank 3: frog is bit rank 3: frog tb its rank 3: frog bi tbs rank 5: rots if big rank 5: robs it fig rank 5: robs it gif rank 5: robt is fig rank 5: robt is gif rank 5: rigs of bot rank 5: rigs ot fob rank 5: rift os gob rank 5: rift os bog rank 5: ribs of tog rank 5: ribs of got rank 5: ribs ot fog rank 0: sift or gob rank 0: sift or bog rank 0: stir of gob rank 0: stir of bog rank 0: stir ob fog rank 6: trig of sob rank 6: trig os fob rank 1: gift rs orb rank 1: gift rb sro rank 1: gift or sob rank 1: gift os rob rank 1: gist or fob rank 1: gist of rob rank 1: gist ob for rank 1: gist rb fro rank 1: girt os fob rank 1: girt of sob rank 1: gris of bot rank 1: gris ot fob rank 1: grit os fob rank 1: grit of sob

Edge Case Tests:

To run the edge case tests, make sure you are in the A2 directory. Then do ./edgeCaseTest.sh in the command line/terminal **(*Note you might have to do chmod +x)**. This will create and write the output to outputs/edgeCaseTest.txt

Tests:

Test #	Expected Output
1	Edge Case Test 1 Input: fortnite Input dictionary: fortnie Encryption dictionary: netfori Encrypted text: netforfi Written to ciphertext.txt Serial Output: No valid words were found from the given dictionary after the decryption(s) Parallel Output: No valid words were found from the given dictionary after the decryption(s).
2	Edge Case Test 2 Input: bruh Input dictionary: bruh Encryption dictionary: hrbu Encrypted text: hrbu Written to ciphertext.txt Serial Output: No valid words were found from the given dictionary after the decryption(s) Parallel Output: No valid words were found from the given dictionary after the decryption(s).
3	Edge Case Test 3 Input: test task! Input dictionary: tesak Encryption dictionary: eastk Encrypted text: ease etsk! Written to ciphertext.txt Serial Output: rank 0: test task! Parallel Output: rank 3: test task!
4	Edge Case Test 4 Input: the cat Input dictionary: theca Encryption dictionary: echta

	Encrypted text: ech tae Written to ciphertext.txt Serial Output: rank 0: eta che rank 0: eat che rank 0: the cat rank 0: the act Parallel Output: rank 0: eta che rank 0: eat che Note: This code only gives two of the possible 4 answers as there are not enough processes to get all 4 answers and brute force all permutations and combinations.

4.2 Performance Analysis

For the performance analysis, I decided to test the parallel program using 3, 5, 7, 9, 11, 12, and 13 unique characters.

To run the test files, make sure you are in the A2 directory. You will see script/.sh file with the respective names of [the number of chars]UniqueCharacters.sh

In the terminal, run this command:
./[# of chars]UniqueCharacters.sh (* replace the [# of chars] with three, five, seven, nine, …)

You will then see the output of the script in a text file in the outputs folder. The text file will have a name of [# of chars]UniqueChars.txt (*Where [# of chars] can be three, five, seven, nine …)

***Note you might need to chmod +x the script file**

Here are the results I got from the different unique character test cases:

Three Unique Characters Data:

All serial times	All parallel times	Average Serial	Average Parallel	Speedup	Efficiency
0.111, 0.116, 0.115, 0.115, 0.109, 0.109,	0.51, 0.49, 0.5, 0.49, 0.51, 0.5, 0.5, 0.5,	0.1111	0.4996667	0.2223482	0.07411608

0.111, 0.107, 0.114, 0.117, 0.109, 0.106, 0.121, 0.103, 0.106, 0.11, 0.117, 0.111, 0.114, 0.113, 0.121, 0.105, 0.108, 0.104, 0.099, 0.115, 0.114, 0.112, 0.113, 0.108	0.49, 0.5, 0.51, 0.5, 0.5, 0.51, 0.49, 0.49, 0.5, 0.49, 0.51, 0.51, 0.5, 0.51, 0.5, 0.5, 0.51, 0.49, 0.51, 0.5, 0.49, 0.48			(Avg. Serial / Avg Parallel)	(Speedup / # of processes which was 3) (*For Parallel)

Five Unique Characters Data:

All serial times	All parallel times	Average Serial	Average Parallel	Speedup	Efficiency
0.099, 0.11, 0.108, 0.124, 0.105, 0.105, 0.102, 0.106, 0.105, 0.111, 0.103, 0.118, 0.104, 0.105, 0.115, 0.095, 0.104, 0.111, 0.1, 0.111, 0.103, 0.103, 0.099, 0.106, 0.114, 0.113, 0.112, 0.104, 0.113, 0.116	0.52, 0.52, 0.53, 0.55, 0.53, 0.53, 0.55, 0.52, 0.51, 0.52, 0.51, 0.51, 0.51, 0.51, 0.51, 0.52, 0.52, 0.51, 0.52, 0.52, 0.52, 0.53, 0.51, 0.53, 0.52, 0.51, 0.52, 0.52, 0.53, 0.57	0.1074667	0.5226667	0.2056122 (Avg. Serial / Avg Parallel)	0.04112245 (Speedup / # of processes which was 5) (*For Parallel)

Seven Unique Characters Data:

All serial times	All parallel times	Average Serial	Average Parallel	Speedup	Efficiency
0.117, 0.119, 0.109, 0.126, 0.117, 0.122, 0.107, 0.109, 0.115, 0.12, 0.116, 0.116, 0.11, 0.112, 0.112, 0.11, 0.104, 0.114, 0.124, 0.121, 0.117, 0.116, 0.114, 0.116, 0.128, 0.115, 0.107, 0.105, 0.119, 0.124	0.56, 0.56, 0.56, 0.58, 0.58, 0.57, 0.6, 0.57, 0.55, 0.55, 0.59, 0.58, 0.57, 0.56, 0.57, 0.56, 0.56, 0.55, 0.58, 0.58, 0.57, 0.56, 0.58, 0.56, 0.58, 0.57, 0.56, 0.58, 0.55, 0.57	0.1153667	0.5686667	0.2028722 (Avg. Serial / Avg Parallel)	0.02898175 (Speedup / # of processes which was 7) (*For Parallel)

Nine Unique Characters Data:

All serial times	All parallel times	Average Serial	Average Parallel	Speedup	Efficiency
0.201, 0.214, 0.207, 0.201, 0.208, 0.199, 0.204, 0.182, 0.216, 0.221, 0.212, 0.24, 0.224, 0.231, 0.21, 0.203, 0.2, 0.195, 0.2, 0.193, 0.204, 0.204, 0.197, 0.206, 0.196, 0.211, 0.2, 0.21, 0.191, 0.207	0.76, 0.78, 0.8, 0.74, 0.8, 0.77, 0.8, 0.73, 1.2, 0.76, 0.76, 0.75, 0.75, 0.75, 0.73, 0.74, 0.74, 0.65, 0.67, 0.68, 0.69, 0.68, 0.7, 0.68, 0.7, 0.69, 0.75, 0.73, 0.67, 0.68	0.2062333	0.7443333	0.2770712 (Avg. Serial / Avg Parallel)	0.03078569 (Speedup / # of processes which was 9) (*For Parallel)

Eleven Unique Characters Data:

All serial times	All parallel times	Average Serial	Average Parallel	Speedup	Efficiency
8.404, 8.322, 8.052, 9.127, 8.204, 8, 8.551, 8.533, 8.503, 9.417, 8.418, 8.511, 8.41, 8.606, 8.214, 8.845, 8.778, 8.381, 8.553, 8.865, 8.44, 8.481, 8.626, 8.191, 8.209, 8.954, 8.346, 8.333, 9.442, 10.481	3.6, 3.67, 3.51, 3.74, 3.73, 1.17, 3.58, 3.75, 3.86, 3.76, 1.24, 4.05, 3.59, 3.64, 3.53, 3.59, 3.64, 3.55, 3.61, 3.55, 3.51, 3.64, 3.53, 3.73, 3.61, 3.82, 4.27, 1.24, 6.35, 6.16	8.606567	3.607333	2.385853 (Avg. Serial / Avg Parallel)	0.2168957 (Speedup / # of processes which was 11) (*For Parallel)

Twelve Unique Characters:

All serial times	All parallel times	Average Serial	Average Parallel	Speedup	Efficiency
60.469, 59.452, 64.325, 64.637, 68.895, 60.899, 60.43, 64.309,	22.58, 23.36, 30.95, 33.21, 34.9, 23.12, 22.95, 33.4, 22.62, 23.23, 22.38, 22.26,	62.23763	24.122	2.580119 (Avg. Serial / Avg Parallel)	0.2150099 (Speedup / # of processes which was

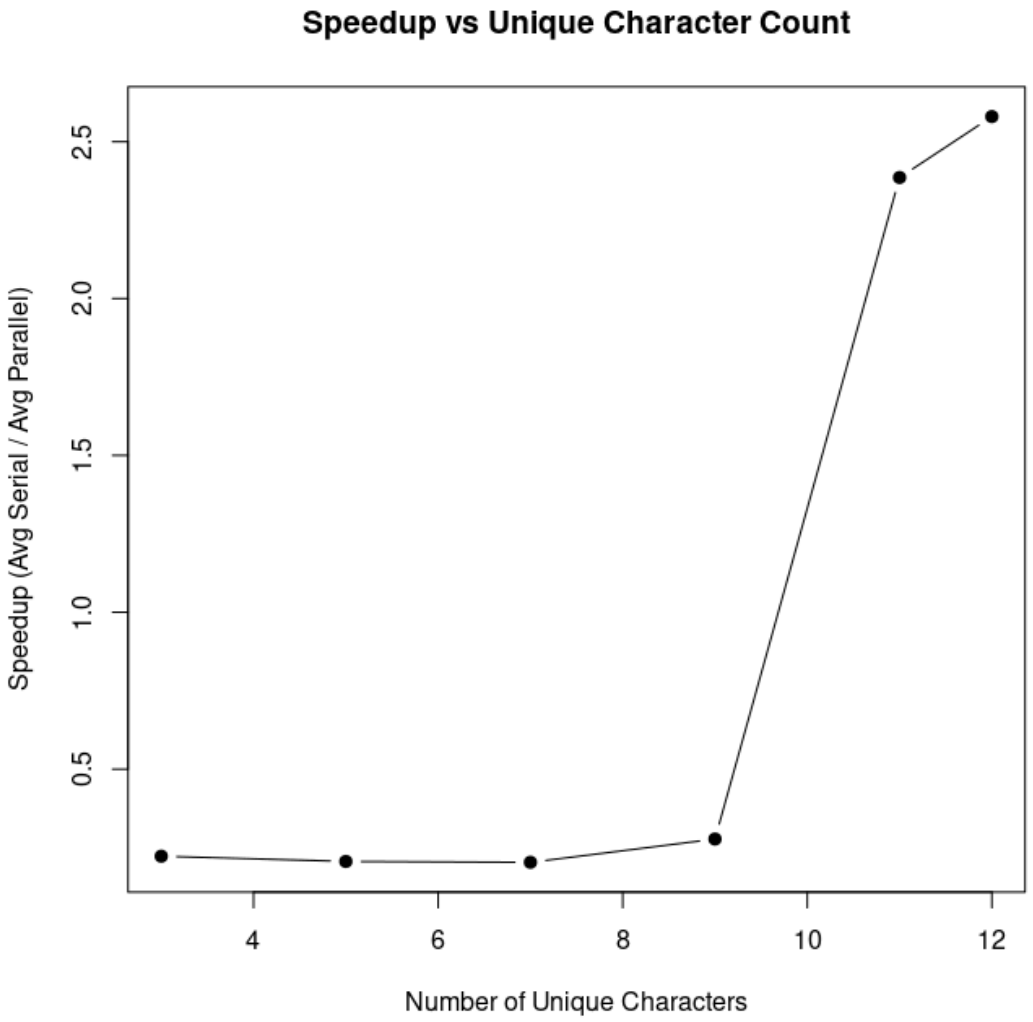
59.898, 62.061, 63.222, 59.679, 61.997, 60.281, 62.517, 59.547, 66.785, 61.219, 65.527, 61.134, 60.425, 61.399, 61.772, 64.328, 61.502, 60.65, 65.566, 60.919, 61.187, 62.098	22.47, 22.13, 22.74, 22.08, 23.69, 22.84, 22.39, 22.86, 22.22, 22.23, 22.65, 22.81, 22.51, 23.81, 22.61, 22.89, 22.75, 23.02,				12) (*For Parallel)

Thirteen Unique Characters:

All serial times	All parallel times	Average Serial	Average Parallel	Speedup	Efficiency
300, 300.005, 300.003, 300.005, 300.005, 300.004, 300.001, 300.005, 300.003, 300.006, 300.005, 300.002, 300.005, 300.005, 300.006, 300.006, 300.005, 300.005, 300.001, 300.004, 300.002, 300.003, 300.003, 300.003, 300.006, 300.006, 300.006, 300.005, 300.005, 300.004,	301.028, 301.036, 301.037, 301.024, 301.035, 301.025, 301.031, 300.009, 301.027, 301.029, 301.04, 301.03, 301.033, 301.038, 301.029, 301.035, 301.033, 301.032, 301.04, 301.025, 301.026, 301.031, 301.037, 301.038, 301.035, 301.033, 1.248 (Note this run got aborted), 301.032, 301.025, 301.029	Timed Out after 5 mins	Timed Out after 5 mins	Timed out after 5 mins	Timed out after 5 mins

Visualizations:

Speedup vs Unique Characters



This is a line graph of the speedup (average serial runtime ÷ average parallel run time) for when testing different amounts of unique characters in the encrypted text.

For smaller unique characters counts of 3, 5, 7, and 9, the serial program was actually faster as the speedup values were around 0.22, 0.21, 0.21, and 0.28 respectively. I suspect this happens because when the search space is small, the overhead cost of parallelism becomes much more expensive than the actual work needed to be done. In my code the parallel version had to:

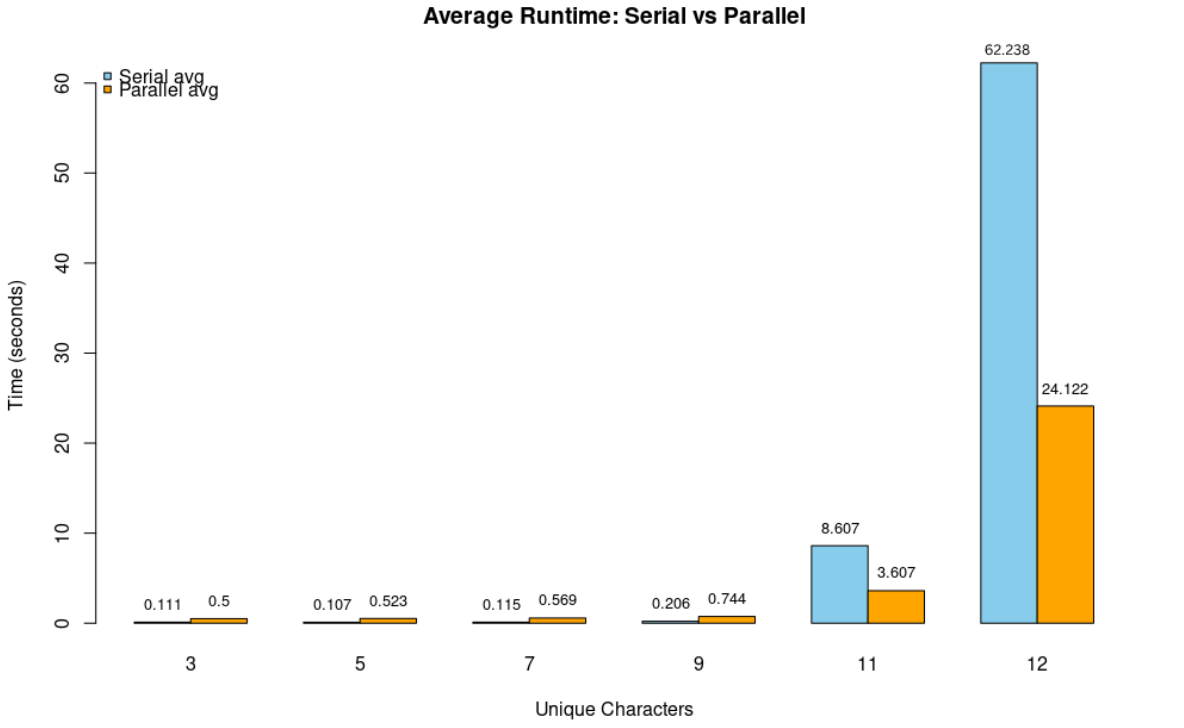
- Initialize the MPI Processes
- Distribute different dictionaries to different processes

- Synchronize result passing, early stopping, and termination signals

When the problem is much smaller, these overheads I suspect can outweigh the actual search time, so the single serial loop can simply brute force the smaller combinations of the cipher text much faster than the parallel system can coordinate and then brute-force them.

However, once the unique characters in the given encrypted string was 11 and 12, the parallel program had a much better speed as it gave a speedup of 2.39 and 2.59 respectively. This is because when there are 11 and 12 unique characters, the search space of the different permutations and combinations of the encrypted string increases massively (factorial growth: 11! ≈ 39 million, 12! ≈ 479 million possibilities). So, now there is finally enough computation in my parallel implementation to write off the MPI overhead. Instead of one process handling all the checks and trying different combinations, dividing the permutations among 11 and 12 processes was much more time efficient and faster.

Avg Serial vs Avg Parallel Run Times:



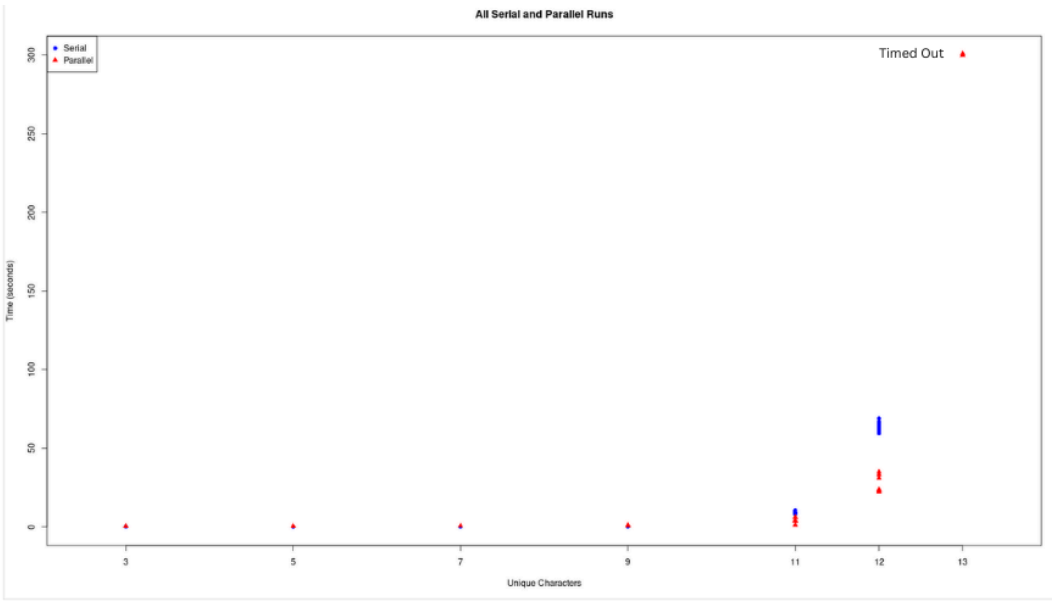
This is a double bar graph of the average serial and average parallel run times in seconds (sky blue for serial and orange for parallel)

As you can see, for unique characters in the encrypted string for (3, 5, 7, and 9) serial was giving a shorter average run time. This is likely due to MPI overhead costs that were discussed earlier on small problem sizes, time spent creating processes, distributing data between processes, and synchronizing results outweighs the benefit of splitting work.

But, when we get to 11 and 12 unique characters, the average runtime is much faster for parallel compared to serial. When we visualize it, there is a very noticeable gap in how fast parallel can get the decrypted strings and complete the program. Again, this is because of the vast amount more permutations the program needs to brute force at 11 and 12 unique characters, hence the computation time being large enough that the MPI overhead time being written off. At that point, dividing the permutations across 11 and 12 processes becomes much more time-efficient, as discussed earlier.

One main conclusion or takeaway from the graph that I see is that even though parallel is slower for the smaller unique character tests, it is not that much slower as it is only slower by 0.4 - 0.5 seconds. But, for the larger unique character tests the serial is much slower and there is a bigger gap in the graph. The serial program takes on average approximately **5** and **38** more seconds to complete for 11 and 12 unique characters! This clearly shows a valuable reason to use parallel programs over the simpler serial programs for much longer tasks as it does give a significant time improvement.

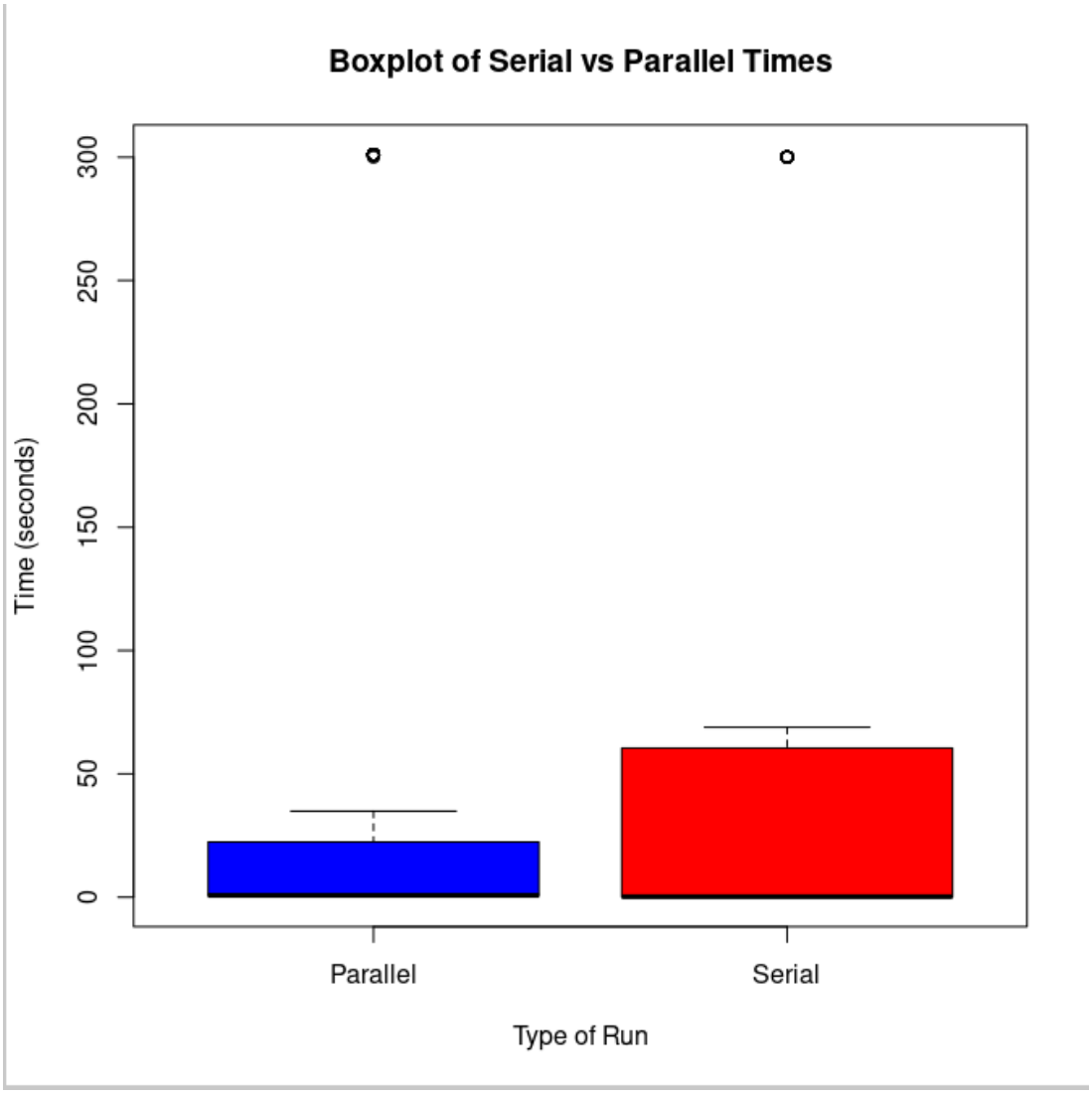
Scatter Plot of all the runs:



This was a scatter plot of all the runs from the different unique characters (sorry for the size, zoom in to get a better view). Again, you can see an overlap of red (parallel) and blue (serial) dots in terms of run times for the smaller unique characters as serial was faster but not by a lot (due to overhead costs). But for 11 and 12 unique chars, the red and blue dots are much more clearly spread out as you can see. The blue dots are much higher compared to the red dots, showcasing the much shorter run times for parallel. Again this is because of the overhead costs being written off, due to the vast amount of permutations to brute force.

*Note: For the thirteen unique character case, serial and parallel were timed out after 5 mins. The red or parallel dots overlap or go a little higher than the parallel because the script timed them out at 5 mins 1 sec, while the serial script was getting timed out at 5 mins exactly.

Boxplot of all runs (serial and parallel):



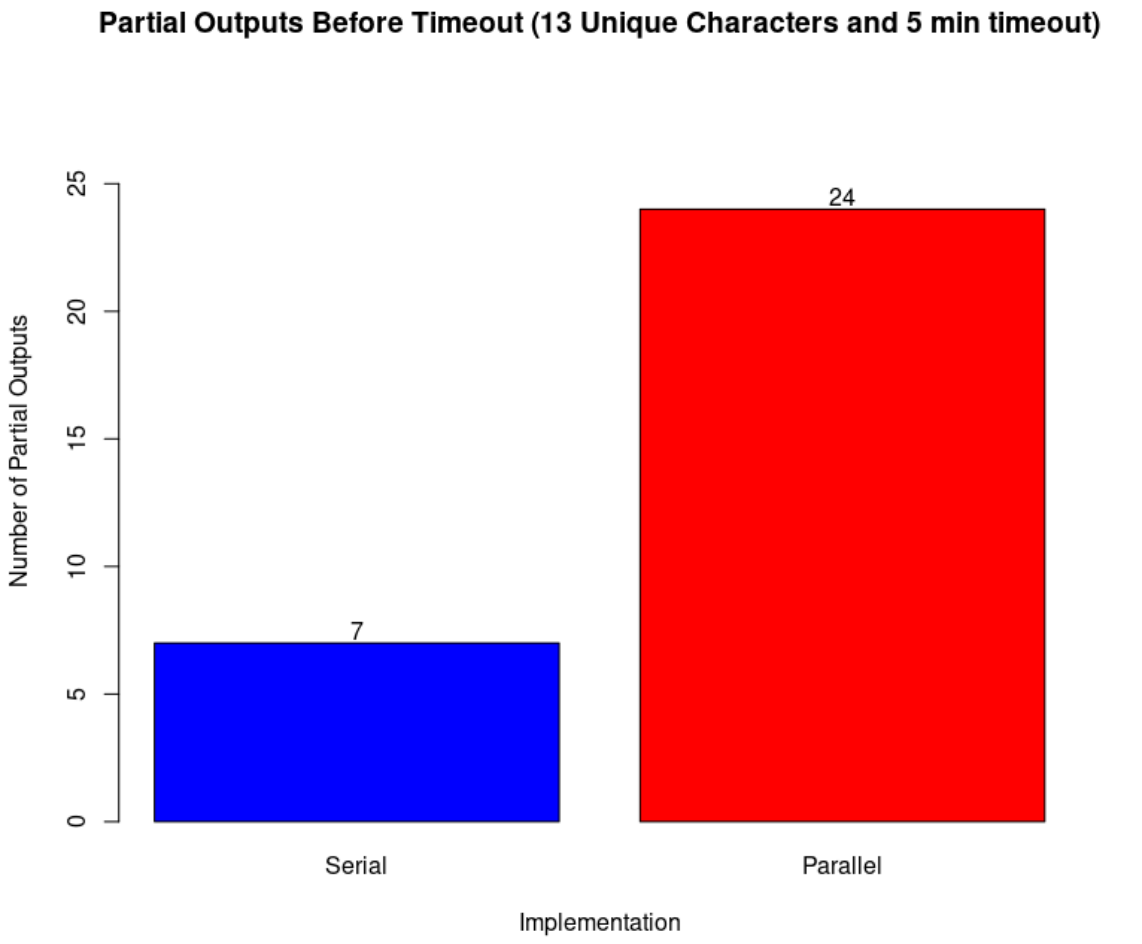
This is a boxplot of all the serial and parallel run times, comparing the distribution of run times for every unique character test.

As you can see, the serial box is much taller and more spread out compared to the parallel one. This indicates that from all the runs we did from 3–12 unique characters (not counting 13, since it is an outlier), the serial program not only took longer on average, but also

showed more variability in how long each run took. The parallel version stayed within a more narrow range because the workload was split across processes, which kept the run times more consistent.

A key takeaway from this boxplot is the exponential growth difference. Serial run times increased from around 0.1–0.2 seconds for small unique character counts to around 8 and 62 seconds for the 11 and 12-character tests. Parallel run times also increased, but much less aggressively — from around 0.5–0.7 seconds for the small tests to about 3 and 24 seconds for 11 and 12 characters. This clearly shows that the parallel program scales much better because the work is divided, making performance more stable, predictable, and consistent as complexity grows.

Partial Output Before Timeout (When time to decrypt is too long 5+ mins):



This bar graph is specifically for the 13-unique-character outlier test case, where the time to decrypt was too long (5+ minutes). For these runs, both serial and parallel implementations had to timeout after 5 minutes. However, when checking the resulting output file

(outputs/thirteenUniqueChars.txt) I observed that the parallel version produced partial decrypted outputs far more often, 24 out of 30 runs (≈80%). Compared to the serial program which only produced 7 out of 30 (≈30.43%).

This makes sense because the work is being divided among 13 processes in the parallel implementation. With such a vast amount of permutations (13!), spreading the brute-force search across multiple processes gives the parallel version a much higher chance of finding a correct decryption before the timeout as multiple processes are testing different paths, whereas a single serial process can only test one path at a time.

Performance Analysis Conclusion

Overall, from all the test cases and visualizations, the main pattern is very clear:

Parallel may not always be faster but it becomes worth it once the problem size is large enough.

For smaller unique-character counts (3, 5, 7, 9) the serial program actually performed slightly better. The difference wasn't huge as it was only a few tenths of a second, but parallelism did not provide a benefit there . This is because of the parallel overhead like initializing MPI, distributing data, synchronizing tasks, and sending termination signals. In those cases, the computation was so small that this overhead outweighed the benefit of splitting the work.

However, once I tested 11 and 12 unique characters, the results were much more different as parallel was much faster compared to serial. For those large cases, the number of permutations increased massively (millions to hundreds of millions of permutations), and parallelism gave a massive improvement. The parallel program scaled much better because it could divide that enormous workload across multiple processes which dramatically reduced runtime compared to serial brute-forcing on a single process. This would also write off any of the MPI or parallel overhead time costs as well.

Even in the extreme 13-character scenario (where both versions hit the 5-minute timeout), parallel still showed a meaningful advantage by producing far more partial outputs before the timeout, as dividing the work allowed more permutations to be explored simultaneously compared to a single serial process.

The final takeaway from all this is that the parallelization MPI does not make everything faster in every case, but once the problem becomes large enough where the computation dominates the cost of overhead, the benefits of parallel programming become extremely clear. In this project, parallel performance started to meaningfully beat serial performance and give huge benefits once the number of permutations to test was large.